CONTROL DEL DOCUMENTO

Proyecto	Sistema de Gestión de Proyectos Ágiles
Versión	0.1.0
Fecha	10 de marzo de 2026
Responsable	Esparza Cortes Cristian Adrián (PM)
Aprobado por	[Nombre del Sponsor/Cliente]
Organización/Cliente	[Nombre de la Organización]
Equipo	Esparza Cortes Cristian Adrian, Esparza Cortes Diego Ricardo, Carrillo Aguirre Gerardo Alejandro, Miranda Viramontes Luis, Rivas Arellano Cesar, Torres Trujillo Luis Octavio

Historial de Cambios

Fecha	Versión	Cambio Principal
10/03/2026	**0.1.0**	**Versión inicial del Plan de Proyecto con asignación de roles**

1. RESUMEN EJECUTIVO

Desarrollar una plataforma web de gestión de proyectos similar a Jira, que permita a los equipos de software organizar, seguir y reportar el avance de sus actividades de manera eficiente. El sistema centralizará la información para reducir tiempos de seguimiento en un 40% (RN01), automatizará reportes para reducir tiempos en un 50% (RN02) y disminuirá errores de coordinación en un 30% (RN03).

Problema u Oportunidad

Las organizaciones actuales sufren por la dispersión de información en múltiples herramientas, la generación manual de reportes y la falta de trazabilidad en los flujos de trabajo, lo que impacta la productividad y la calidad del software entregado.

Solución Propuesta

Una aplicación web centralizada que incluya:

- Tableros Kanban y gestión de Sprints.
- Gestión de Epics, Historias de Usuario y Tareas.
- Roles diferenciados (Admin, PO, PM, Dev, Tester).
- Reportes automáticos (Gantt, Velocity, Carga de Trabajo).
- Trazabilidad completa de bugs y cambios.

Alcance y Entregables Principales

- **Módulo de Seguridad:** Autenticación, roles y auditoría (CU01, EP02).
- **Módulo de Gestión:** Tableros, workflows y backlogs (CU02, CU03, CU07, EP01, EP03).
- **Módulo de Calidad:** Gestión de bugs y pruebas (CU06).
- **Módulo de Reportes:** Métricas y dashboards para PM (CU04, EP04).
- **Documentación:** Manuales de usuario, técnico y cierre según ISO/IEC 29110.

Calendario de Alto Nivel

- **Duración Total:** 4 semanas.
- **Sprints:** 4 iteraciones de 1 semana cada una.
- **Metodología:** Híbrido (Scrum + TSP + Design Thinking) alineado a ISO/IEC 29110 Perfil Básico.

Costos Estimados

- **Licencias:** $0 USD (Open Source: .NET, SQL Server Express, Git).
- **Recursos:** 6 desarrolladores × 4 semanas = 720 horas totales.
- **Infraestructura:** Equipos personales y conexión a internet.

Riesgos Críticos

1. Alcance excesivo para 4 semanas (Nivel: Alto).
2. Complejidad técnica en reportes automáticos (Nivel: Medio).
3. Curva de aprendizaje de herramientas nuevas (Nivel: Bajo).

Criterios de Éxito

✓ Cumplimiento de RN01, RN02 y RN03 (validados post-implementación).
✓ 100% de Epicas (EP01-EP04) implementadas.
✓ 0 bugs críticos en la versión 1.0.
✓ Documentación completa según estándar

---

2. CONTEXTO, PROPÓSITO Y JUSTIFICACIÓN

Contexto Institucional

El desarrollo de software requiere herramientas robustas para gestionar la complejidad. Existencia de múltiples soluciones en el mercado (Jira, Trello), pero la necesidad de una solución adaptada a los procesos específicos de la organización y los requerimientos de negocio (RN01-RN03) justifica el desarrollo propio.

Necesidad que Origina el Proyecto

- **RN01:** Centralizar información para disminuir tiempo de seguimiento en 40%.
- **RN02:** Automatizar reportes para reducir tiempo de generación en 50%.
- **RN03:** Estandarizar comunicación para disminuir errores en 30%.

Justificación Técnica y de Negocio

Negocio: La eficiencia en la gestión de proyectos impacta directamente en el time-to-market y la calidad del producto final. **Técnica:** Se utiliza un enfoque híbrido:

- **Design Thinking:** Para asegurar que la herramienta resuelva los dolores reales de los usuarios (CU01-CU07).
- **Scrum:** Para entregar valor incrementalmente en 4 sprints.
- **TSP:** Para garantizar disciplina en la estimación, calidad y métricas del equipo de desarrollo.
- **ISO/IEC 29110:** Para asegurar que el proceso de desarrollo esté estandarizado y sea auditable.

Valor Esperado

- Plataforma propietaria adaptable a necesidades específicas.
- Mejora en la visibilidad del estado de los proyectos.

- Reducción de carga administrativa para Project Managers y Desarrolladores.

---

3. OBJETIVOS DEL PROYECTO

3.1 Objetivo General

Desarrollar e implementar un sistema de gestión de proyectos ágiles (tipo Jira) en 4 semanas que permita centralizar información, automatizar reportes y mejorar la coordinación del equipo, cumpliendo con los RN01, RN02 y RN03,

3.2 Objetivos Específicos

1. **Implementar núcleo de seguridad y usuarios** (CU01, EP02) en el Sprint 1.
2. **Desarrollar gestión de tickets y tableros** (CU02, CU03, CU07, EP01, EP03) en el Sprint 2.
3. **Implementar flujo de calidad y bugs** (CU05, CU06) en el Sprint 3.
4. **Desarrollar módulo de reportes y métricas** (CU04, EP04) en el Sprint 4.

3.3 Indicadores y Metas

Indicador	Meta	Criterio de Logro	Método de Medición
Cumplimiento de Scope	100%	Todas las EP implementadas	Revisión Sprint 4
Calidad de Código	0 Bugs Críticos	Sin fallos bloqueantes	Pruebas de Aceptación
Productividad	88 Puntos Historia	Total backlog completado	Tablero Scrum
Satisfacción Usuario	≥4/5	Usuarios felices con la herramienta	Encuesta Post-Release

---

4. ALCANCE DEL PROYECTO

4.1 Alcance Incluido

- **Gestión de Identidades (CU01):** Login, roles, auditoría, bloqueos.

- **Configuración de Workflows (CU02):** Estados personalizados, permisos, campos.
- **Gestión de Backlog (CU03):** Epics, Historias, Priorización (Drag & Drop).
- **Seguimiento (CU04):** Hitos, Reportes (Gantt, Velocity), Riesgos.
- **Calidad (CU05, CU06):** Revisión técnica, gestión de bugs, pruebas.
- **Desarrollo (CU07):** Asignación de tareas, registro de tiempo, codificación.
- **Tecnología:** .NET 8, SQL Server, Web API, Frontend Responsive.

4.2 Fuera de Alcance

- Aplicación móvil nativa (iOS/Android).
- Integración con herramientas externas (GitHub, Jira externo, Slack).
- Inteligencia Artificial para predicción de tiempos.
- Multi-idioma (Solo Español).
- Alta disponibilidad empresarial (99.99%).

4.3 Supuestos y Restricciones

- **Tiempo:** 4 semanas exactas (4 sprints de 1 semana).
- **Costo:** Sin presupuesto para licencias o nube paga.
- **Recursos:** 6 integrantes disponibles full-time (30h/semana c/u).
- **Tecnología:** Stack Microsoft (.NET) y SQL Server.
- **Normatividad:** Cumplimiento ISO/IEC 29110 Perfil Básico.

4.4 Entregables y Criterios de Aceptación

Entregable	Criterio de Aceptación	Responsable	Sprint
Sistema Autenticación	Login seguro, roles funcionales	Diego (Tech Lead)	1
Tablero Kanban	Mover tickets, estados configurables	Equipo	2
Gestión de Bugs	Reportar y vincular bugs a HU	Cesar (QA)	3
Reportes Automáticos	Gantt y Velocity generados	Cristian (PM)	4
Documentación	Manuales completos y código limpio	Luis O. (Config)	4

---

5. ENFOQUE METODOLÓGICO Y GOBERNANZA DEL HÍBRIDO

5.1 Principios de Integración

- **Design Thinking:** Empathize, Define, Ideate, Prototype, Test (Centrado en CU01-CU07).
- **Scrum:** Sprints de 1 semana, Daily, Review, Retro.
- **TSP:** Roles definidos, medición de tiempo y defectos, revisiones formales.
- **ISO/IEC 29110:** Procesos PM (Gestión) y SR (Realización) documentados.

5.2 Fases y Puntos de Control

- **Inicio (Semana 0):** Definición de roles TSP, Backlog inicial, Arquitectura.
- **Construcción (Sprints 1-4):** Desarrollo iterativo, revisiones de código, pruebas.
- **Cierre (Semana 5):** Despliegue, capacitación, lecciones aprendidas.

---

6. ORGANIZACIÓN DEL EQUIPO, ROLES Y RESPONSABILIDADES

6.1 Roles del Proyecto (Híbridos)

Rol	Responsable	Funciones Principales
Project Manager (TSP)	**Esparza Cortes Cristian Adrian**	- Planificación y control del proyecto. - Reportes a stakeholders. - Desarrollo de funcionalidades.
Product Owner (Scrum)	**Miranda Viramontes Luis**	- Gestión del Backlog. - Priorización por valor (RN01-RN03). - Validación de criterios de aceptación. - Desarrollo de funcionalidades.
Scrum Master	**Carrillo Aguirre Gerardo Alejandro**	- Facilitación de ceremonias. - Eliminación de impedimentos. - Desarrollo de funcionalidades.
Tech Lead	**Esparza Cortes Diego Ricardo**	- Arquitectura y estándares de código. - Revisiones técnicas (Code Review).

Rol	Responsable	Funciones Principales
		- Resolución de problemas técnicos. - Desarrollo de módulos críticos.
QA / Calidad (TSP)	**Rivas Arellano Cesar**	- Plan de pruebas. - Ejecución de pruebas y reporte de bugs. - Desarrollo de funcionalidades.
Configuración (TSP)	**Torres Trujillo Luis Octavio**	- Control de versiones (Git). - Despliegue y ambientes. - Desarrollo de funcionalidades.

6.2 Matriz RACI

Actividad/Entregable	Sponsor	PO (Luis M.)	SM (Gerardo)	PM (Cristian)	Tech Lead (Diego)	Equipo (Cesar, Luis O.)
Requisitos (SRS)	A	R	C	C	C	I
Arquitectura (SDS)	I	C	I	A	R	C
Código Fuente	I	I	I	A	R	R
Pruebas	I	C	I	C	C	R (Cesar)
Reportes de Avance	R	C	C	R	I	I
Despliegue	I	C	I	A	C	R (Luis O.)

(A=Obligado, R=Responsible, C=Consultado, I=Informado)

---

7. ALINEACIÓN A ISO/IEC 29110 (PERFIL BÁSICO)

7.1 Cobertura PM (Gestión del Proyecto)

- **Planeación:** Secciones 1, 3, 4, 10, 18 de este plan.
- **Monitoreo y Control:** Informes de avance semanales (Sección 17).
- **Gestión de Riesgos:** Registro de riesgos (Sección 16).

- **Cierre:** Acta de aceptación y lecciones aprendidas (Sección 20).

7.2 Cobertura SR (Realización de Software)

- **Requisitos:** Especificación basada en RN, CU y EP (Sección 12).
- **Diseño:** Arquitectura y diagramas UML (Sección 13).
- **Construcción:** Código fuente en repositorio Git (Sección 14).
- **Pruebas:** Plan de pruebas y reporte de bugs (Sección 15).
- **Entrega:** Manual de usuario e instalación (Sección 19).

---

8. PLAN DE DESIGN THINKING (DESCUBRIMIENTO Y VALIDACIÓN)

8.1 Alcance de Investigación

- **Personas:** Administrador (CU01), PM (CU04), Dev (CU07), Tester (CU06).
- **Journey:** Flujo desde la creación de una HU hasta su despliegue.
- **Puntos de Dolor:** Reportes manuales, falta de trazabilidad, comunicación dispersa.

8.2 Técnicas y Evidencia

- **Entrevistas:** A representantes de cada rol (CU).
- **Prototipado:** Wireframes de tableros y reportes.
- **Pruebas de Usabilidad:** Validación de flujos en cada Sprint Review.

8.3 Artefactos de Salida

- Mapas de empatía por rol.
- Problem Statement: "Los equipos pierden 40% del tiempo en seguimiento manual".
- Backlog inicial priorizado (EP01-EP04).

---

9. PLAN SCRUM (GESTIÓN DEL PRODUCTO Y EJECUCIÓN ITERATIVA)

9.1 Backlogs y Definiciones

- **Product Backlog:** Basado en EP01-EP04 y CU01-CU07.

- **Sprint Backlog:** Tareas técnicas definidas en Planning.
- **DoR (Definition of Ready):** HU clara, criterios de aceptación definidos.
- **DoD (Definition of Done):** Código revisado, pruebas passing, documentado.

9.2 Cadencia y Eventos

- **Sprints:** 4 semanas (1 semana c/u).
- **Daily:** 15 min (Lunes a Viernes).
- **Planning:** 2 horas (Inicio de Sprint).
- **Review:** 1 hora (Fin de Sprint).
- **Retro:** 1 hora (Fin de Sprint).

9.3 Transparencia y Reportes

- **Tablero:** Kanban digital (To Do, In Progress, Review, Done).
- **Métricas:** Burn-down chart de Sprint.
- **Incremento:** Funcionalidad ejecutable al final de cada semana.

---

10. PLAN TSP (ESTIMACIÓN, MEDICIÓN, CALIDAD Y CONTROL)

10.1 Método de Estimación y Capacidad

- **Unidad:** Puntos de Historia (Escala Fibonacci).
- **Total Proyecto:** 88 Puntos.
- **Capacidad:** 180 horas por Sprint (6 personas × 30 horas).
- **Conversión:** 1 Punto ≈ 4 Horas.

10.2 Plan de Medición (Métricas Mínimas)

- **Esfuerzo:** Plan vs Real (Horas).
- **Defectos:** Inyección y Remoción por fase.
- **Productividad:** Puntos completados por Sprint.
- **Calidad:** Densidad de defectos.

10.3 Revisiones e Inspecciones

- **Revisión de Código:** Obligatoria antes de merge (Responsable: Diego).
- **Checklists:** Requisitos, Diseño, Código, Pruebas.

---

11. GESTIÓN DE CONFIGURACIÓN Y CONTROL DE CAMBIOS

11.1 Control de Versiones

- **Repositorio:** GitHub/GitLab.
- **Ramas:** main (producción), develop (integración), feature/sprint-X.
- **Tags:** v0.1.0, v0.2.0, v0.3.0, v1.0.0.
- **Responsable:** Torres Trujillo Luis Octavio.

11.2 Gestión de Cambios

- **Solicitud:** Issue en repositorio.
- **Análisis:** Impacto en tiempo y alcance (PM + PO).
- **Aprobación:** Sponsor/Cliente para cambios críticos.

---

12. GESTIÓN DE REQUISITOS Y TRAZABILIDAD

12.1 Estrategia

- Levantamiento mediante Design Thinking.
- Formalización en Historias de Usuario.

12.2 Matriz de Trazabilidad

- RN → CU → EP → HU → Código → Prueba.
- **Ejemplo:** RN01 → CU04 → EP04 → HU08 (Gantt) → Prueba CT04.

12.3 Criterios de Aceptación

- Definidos por cada HU en el Backlog.
- Validados por el PO (Luis Miranda).

---

13. ARQUITECTURA Y DISEÑO (SDS)

13.1 Decisiones de Arquitectura

- **Patrón:** MVC + Repository.
- **Backend:** .NET 8 Web API.
- **Frontend:** Razor Pages / Blazor / Bootstrap.
- **BD:** SQL Server Express.
- **Auth:** JWT Tokens.

13.2 Modelado

- **Entidades:** Usuario, Proyecto, Ticket, Epic, Sprint, Bug.
- **Diagramas:** Clases, Contexto, Secuencia (Responsable: Diego).

13.3 Requisitos No Funcionales

- **Seguridad:** Encriptación BCrypt, HTTPS.
- **Rendimiento:** Respuesta < 2 segundos.
- **Usabilidad:** Interfaz intuitiva tipo Jira.

---

14. DESARROLLO E INTEGRACIÓN

14.1 Estándares de Codificación

- **Lenguaje:** C# 12.
- **Convenciones:** PascalCase (clases), camelCase (variables).
- **Comentarios:** XML en métodos públicos.

14.2 Estrategia de Integración

- **CI:** Build automático en push a develop.
- **Ambientes:** Local (Dev), Servidor Pruebas (QA), Localhost (Prod Demo).

---

15. PRUEBAS Y ASEGURAMIENTO DE CALIDAD

15.1 Estrategia de Pruebas

- **Unitarias:** xUnit (Min 80% cobertura).

- **Integración:** APIs y BD.
- **Sistema:** Flujos completos (E2E).
- **Aceptación:** Validación por PO.

15.2 Gestión de Defectos

- **Severidad:** Crítico, Alto, Medio, Bajo.
- **Ciclo:** Reporte (Cesar) → Asignación → Corrección → Re-prueba.

---

16. GESTIÓN DE RIESGOS

16.1 Registro y Evaluación

ID	Riesgo	Probabilidad	Impacto	Estrategia	Responsable
R01	Alcance excesivo para 4 semanas	Alta	Alto	Mitigar (Priorizar Backlog)	Cristian (PM)
R02	Bugs críticos en producción	Media	Alto	Transferir (Pruebas exhaustivas)	Cesar (QA)
R03	Cambios de requisitos	Alta	Medio	Mitigar (Congelar por Sprint)	Luis (PO)
R04	Errores de coordinación (RN03)	Alta	Alto	Mitigar (Daily + Tablero)	Gerardo (SM)

16.2 Respuestas

- **Mitigar:** Acciones proactivas para reducir probabilidad/impacto.
- **Aceptar:** Monitorear sin acción inmediata (riesgos bajos).

---

17. COMUNICACIÓN Y SEGUIMIENTO

17.1 Stakeholders

- **Sponsor:** Reporte Ejecutivo Semanal.
- **Equipo:** Daily, Tablero, Retro.

- **Usuarios Finales:** Demo al final de cada Sprint.

17.2 Canales

- **Reuniones:** Presenciales/Virtuales según agenda.
- **Documentación:** Carpetas compartidas (ISO 29110).
- **Mensajería:** Grupo de equipo para incidencias rápidas.

---

18. RECURSOS Y COSTOS

18.1 Capacidad del Equipo

- **Total:** 720 horas (6 personas × 120 horas c/u).
- **Distribución:** 30 horas semanales por persona.

18.2 Herramientas

- **IDE:** Visual Studio 2022 Community.
- **BD:** SQL Server Express.
- **Control:** Git.
- **Costo Total:** $0 USD (Licencias Gratuitas).

---

19. DESPLIEGUE, ENTREGA Y TRANSICIÓN

19.1 Estrategia de Release

- **v0.1.0:** Sprint 1 (Auth).
- **v0.2.0:** Sprint 2 (Tableros).
- **v0.3.0:** Sprint 3 (Bugs/Flujo).
- **v1.0.0:** Sprint 4 (Reportes/Release).

19.2 Documentación y Capacitación

- **Manuales:** Usuario y Técnico.
- **Capacitación:** Sesión de 2 horas con usuarios clave.

Este punto lo lleno con las necesidades esperadas

20. CIERRE Y LECCIONES APRENDIDAS

20.1 Acta de Aceptación

- Firmada por PO y Sponsor al cumplir criterios de RN01-RN03.

20.2 Retrospectiva Final

- **Qué funcionó:** Daily puntuales, Code Reviews. (Lo esperado)
- **Qué mejorar:** Estimación de tareas, automatización de pruebas. (Lo esperado)
- **Acciones:** Documentar durante el desarrollo, no al final. (Obligatorio)

20.3 Archivo Final

- Código fuente, BD, Documentación ISO, Plan de Proyecto.